MULTI-TASK LEARNING / EXPERIMENT REPORT

多任务学习架构对比实验总结

独立基线 · 共享主干 + 任务头 · A-Net Top-1 硬路由专家

实验日期：**2026-10-29** 主实验环境：PyTorch 2.15.0+cpu / Python 3.14.6 / AMD Ryzen 9 9955HX / 单线程 CPU

CUDA 验证：PyTorch 2.14.1+cu130 / CUDA 13.0 / NVIDIA GeForce RTX 5060 Laptop GPU

核心结论

- **准确率接近饱和。**独立基线 99.50%、共享主干 99.42%、A-Net 99.37%，最大差距仅 0.13 个百分点；在当前任务上，准确率不足以单独决定选型。
- **共享主干最省参数。**仅 4,742 个可训练参数，为独立基线的 35.3%，平均准确率只下降 0.08 个百分点。
- **A-Net 节省的是单样本计算，不是总存储。**总参数 6,630，但每个样本只激活 2,338 个参数（共享 L1 + 1 个专家）；CPU batch=256 时延迟 74.2 µs，比独立基线快约 15.8%，吞吐高约 18.6%。
- **延迟具有明显的 batch 依赖。**CPU batch=1 时 A-Net 为 45.0 µs，反而比独立基线 41.8 µs 慢约 7.7%；硬路由的收益在批量推理、计算占主导时才显现。
- **复杂任务拉开差距。**复杂任务上独立基线 96.23%、共享主干 94.49%、A-Net 95.84%；A-Net 仅比独立低 0.39 个百分点，但总参数为独立基线的 49.6%，单样本激活参数只有 17.0%。

注：准确率为测试集 Top-1 准确率；延迟为纯模型 forward 的 P50，不含数据加载与预处理。

补充验证 在 RTX 5060 Laptop 上，CUDA 兼容性测试全部通过；GPU 50 epoch 准确率与 CPU 完全一致。但此超小模型在 batch=1 和训练场景中没有获得 GPU 加速，详见第 5 节。

1 实验设置

实验使用三个二维合成二分类任务，每个任务独立生成训练集和测试集，总样本量 9,600。三个任务的输入维度相同（2 维），但决策边界分别为正弦曲线、圆形和异或形态，用于观察共享表示与任务专属专家在不同非线性结构下的表现。

1.1 三个 2D 非线性任务

表 1 合成任务定义与数据生成方式

任务	标签规则	数据生成	测试集特点
任务 1 正弦边界	x2 > sin(2·x1) 为正类	x1、x2 ~ U(-2, 2)	周期型非线性边界，类别接近平衡
任务 2 圆形边界	r < 1.0 为类别 0，r > 1.0 为类别 1	极坐标分层采样：内圈 r∈[0, 0.95]，外圈 r∈[1.05, 2.0]	圆形边界，类别严格平衡
任务 3 XOR	x1·x2 > 0 为正类	x1、x2 ~ U(-2, 2)	四象限异或，类别严格平衡

每个任务生成 2,400 个训练样本和 800 个测试样本，训练集与测试集使用独立随机流。圆形任务采用平衡的内外圈采样，避免均匀方形采样导致圆内样本只占约 19.6% 的类别不平衡问题。

1.2 模型结构与训练配置

三种结构均接受二维输入和 Task ID。独立基线为每个任务训练一个完整 MLP；共享主干方案让三个任务共用两层特征提取器，只保留任务专属的线性分类头；A-Net 让三个任务共享一个 L1 浅层接口，并为每个任务配置一个独立专家，通过 Task ID 做 Top-1 硬路由。

表 2 三种模型结构与参数口径

方案	结构	总参数	单样本激活参数	计算路径
独立基线	3 × MLP：2-64-64-2	13,446	4,482	按 Task ID 选择一个完整 MLP
共享主干	共享 2-64-64 主干 + 3 个线性头	4,742	4,742	全部主干参数参与每个样本的前向
A-Net	共享 L1：2-64；3 个专家：64-32-2	6,630	2,338	L1 + 仅一个 Task ID 对应专家

训练统一使用 Adam 优化器，学习率 1e-3，训练 50 个 epoch，batch size 为 128，随机种子为 42。每个 epoch 依次遍历三个任务，每个任务内部独立打乱；所有模型使用相同的数据、优化器、batch 大小和 epoch 数，并在训练前重置随机种子。实验限制 PyTorch 为单线程，以符合单核 CPU 的测试目标。

A-Net 的硬路由实现不遍历专家列表，也不做 all-experts 拼接：forward 中直接取 self.experts[task_id] 并只调用该专家。因此未被选中的两个专家既不参与前向计算，也不会收到当前 batch 的梯度；总参数仍包含三个专家，用于衡量存储成本。

2 测试结果：准确率与参数效率

2.1 准确率对比

三个模型在测试集上的 Top-1 准确率如下。任务 2 的圆形边界对三种结构都过于容易，全部达到 100%；差异主要出现在正弦任务和 XOR 任务上，但整体都处于接近饱和的区间。

表 3 测试集准确率

方案	任务 1 正弦	任务 2 圆形	任务 3 XOR	平均准确率
独立基线	99.25%	100.00%	99.25%	99.50%
共享主干 +3 个线性头	99.25%	100.00%	99.00%	99.42%
A-Net 极简版（硬路由）	98.75%	100.00%	99.37%	99.37%

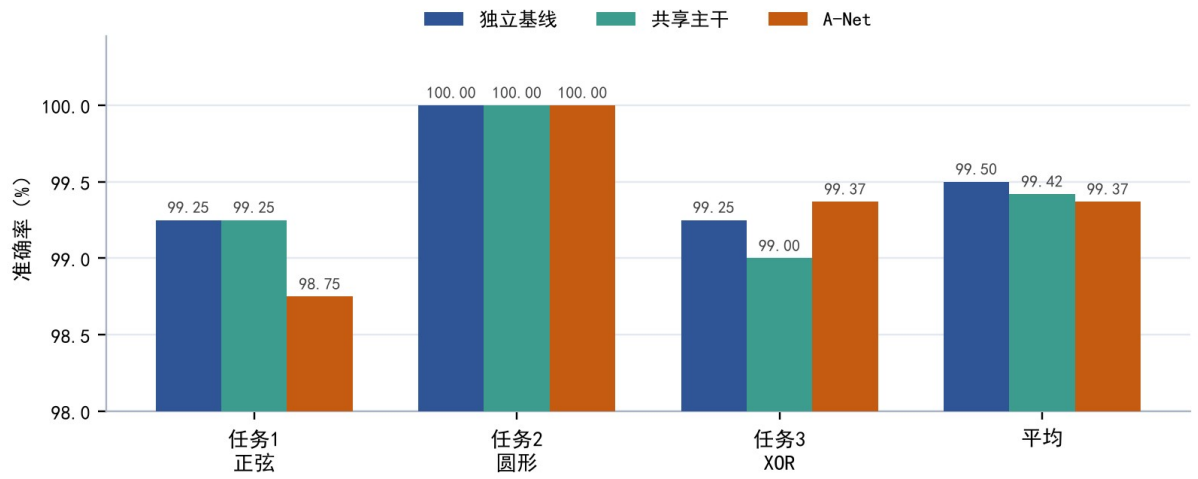


图 1 三个任务及平均准确率对比（y 轴从 98% 起，以突出差异）

独立基线取得最高的平均准确率 99.50%，但只比共享主干高 0.08 个百分点，比 A-Net 高 0.13 个百分点。由于任务较简单、单次固定种子实验且结果接近 100%，这点差异不足以证明某一结构具有统计显著优势。更合理的解读是：三种结构都有足够容量拟合当前任务。

A-Net 在正弦任务上为 98.75%，是三者中最低的；它在 XOR 任务上为 99.37%，略高于共享主干。这与专家的容量和共享 L1 的表达约束有关，也说明“硬路由”并不会自动提升精度，路由结构应根据任务相关性和数据规模选择。

2.2 参数效率分析

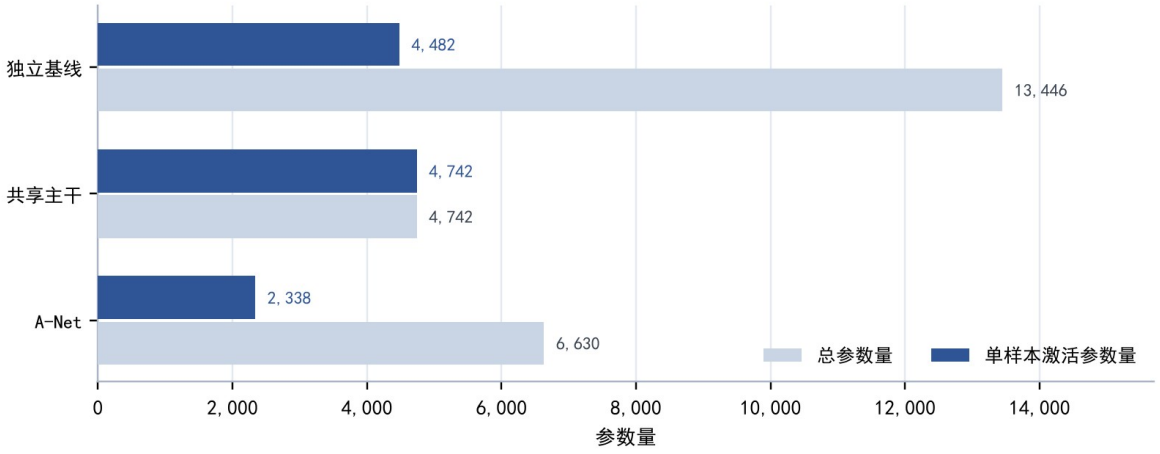


图 2 总参数量与单样本激活参数量对比

共享主干的参数效率最高。它把总参数从 13,446 压缩到 4,742，仅为独立基线的 35.3%，平均准确率却只下降 0.08 个百分点。但需要强调，共享主干的总参数和激活参数相同：每个样本仍要经过完整主干，因此参数减少主要体现为存储和内存占用，而不是前向计算量下降。

A-Net 的总参数为 6,630，是独立基线的 49.3%。这些参数包含三个完整专家，但单样本只激活 L1 加一个专家，共 2,338 个参数，约为独立基线激活参数的 52.2%。这揭示了两个不同口径：总参数衡量模型需要保存多少权重，激活参数衡量一次推理实际执行多少计算。A-Net 用更大的总存储换取了更小的单样本计算路径。

因此，不能简单地说“共享主干比 A-Net 更轻”。如果比较部署包大小和内存占用，共享主干更优；如果比较已知 Task ID 时的单样本计算量，A-Net 的激活路径更短。两者的取舍取决于服务形态和硬件约束。

3 推理延迟与吞吐

3.1 基准方法

延迟测试只测量模型 forward，不包含数据加载、序列化、网络传输和预处理。硬件为 AMD Ryzen 9 9955HX，PyTorch 2.15.0+cpu，Python 3.14.6，PyTorch 与 MKL/OpenMP 均限制为单线程；使用 inference_mode，随机初始化权重。延迟由网络结构和稠密计算图决定，训练权重不会改变这些算子形状。

每个配置先做 warmup，再分多轮连续调用；模型顺序在轮间轮换，以降低 CPU 频率漂移带来的偏差。batch=1 使用 7 轮、每轮 3,000 次，batch=64/256 使用 5 轮；报表以 P50 为主，P95 用于观察尾延迟。吞吐按“处理的样本数 / 总耗时”计算。

3.2 结果与解释

表 4 CPU 推理延迟与 batch=256 吞吐（P50）

方案	激活参数	batch=1	batch=64	batch=256	batch=256 吞吐
独立基线	4,482	41.8 µs	54.7 µs	88.1 µs	2.86 M/s
共享主干 +3 个线性头	4,742	42.4 µs	55.3 µs	88.3 µs	2.85 M/s
A-Net 极简版（硬路由）	2,338	45.0 µs	53.4 µs	74.2 µs	3.39 M/s

表 4 中的延迟是整个 batch 一次 forward 的耗时；batch=1 即为单样本延迟；M/s 表示百万样本/秒。

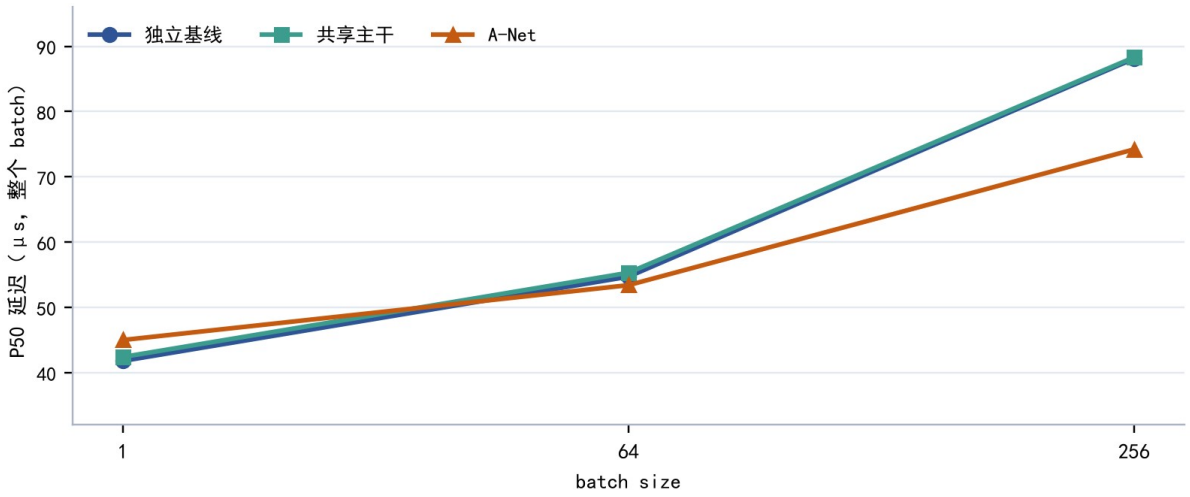


图 3 P50 延迟随 batch size 的变化

batch=1：框架开销占主导，A-Net 反而最慢。独立基线 41.8 µs、共享主干 42.4 µs、A-Net 45.0 µs。A-Net 的激活参数只有 2,338，但额外的路由校验、L1 与专家两个模块边界、Python eager 调度开销超过了缩小矩阵带来的收益，相对独立基线慢约 7.7%。这说明“参数少”不等于“单请求一定快”。

batch=64：A-Net 开始小幅领先。A-Net 为 53.4 µs，独立基线为 54.7 µs，约快 2.4%；差距仍然很小，说明该 batch 下调度开销与计算量大致相当。

batch=256：硬路由的收益变得明确。A-Net 为 74.2 µs，独立基线为 88.1 µs，降低约 15.8%；共享主干为 88.3 µs，几乎与独立基线相同。A-Net 的吞吐达到 3.39 M samples/s，比独立基线的 2.86 M/s 高约 18.6%。batch=256 的 P95 分别为 96.6、96.8、82.9 µs，A-Net 的尾延迟也明显更低。

共享主干和独立基线的延迟几乎相同，尽管前者总参数少 65%。原因是两者的单样本激活参数分别为 4,742 和 4,482，计算路径长度和矩阵规模非常接近。这个结果再次说明：评估推理延迟必须看激活路径与实际 batch，而不是只比较总参数量。

在 batch=1 时，A-Net 的 P95 为 51.2 µs，也略高于独立基线的 47.4 µs 和共享主干的 47.2 µs。对极低延迟的单请求场景，若要发挥 A-Net 的计算优势，需要进一步使用 TorchScript、ONNX Runtime、C++ 部署或算子融合，把 Python 层调度成本压下去；否则硬路由的收益会被运行时开销抵消。

4 结论与选型建议

4.1 核心结论

- **没有单一最优架构。**独立基线精度最高但总参数最大；共享主干参数效率最好但延迟不降；A-Net 总存储居中、单样本激活计算最少，但收益依赖 batch 和运行时。
- **总参数量与延迟必须分开看。**总参数决定存储、加载和内存占用；激活参数决定前向计算量。共享主干减少了前者，A-Net 减少了后者。将总参数量直接当作延迟指标会得出错误结论。
- **Top-1 硬路由适合批量多任务推理。**当 Task ID 已知、batch 足够大、计算占主导时，A-Net 的 64→32 专家比 64→64 MLP 更省算力，在 CPU batch=256 下取得约 15.8% 的延迟优势和约 18.6% 的吞吐优势。
- **小 batch 首先应优化运行时，而不是继续缩小网络。**CPU batch=1 时三者都在 42–45 µs 量级，框架调度成本远大于矩阵计算差异；此时 TorchScript/ONNX/C++ 的收益可能比更换架构更直接。
- **CUDA 兼容，但 GPU 不是这个超小模型的加速捷径。**RTX 5060 Laptop + PyTorch 2.14.1+cu130 的全部测试通过，GPU 50 epoch 准确率与 CPU 完全一致；但 GPU batch=1 约 71–77 µs，慢于 CPU 的 42–45 µs，完整训练也受 CUDA 启动和 kernel 调度开销影响。
- **任务越复杂，A-Net 的性价比越明显。**复杂任务上 A-Net 平均 95.84%，与独立基线的 96.23% 只差 0.39 个百分点，却只需要一半总参数和 17.0% 的激活参数；共享主干降至 94.49%，且跨种子波动明显更大。

4.2 场景化选型

表 5 不同部署约束下的推荐方案

场景	首选方案	关键依据	注意事项
单请求、极低延迟	独立基线	batch=1 最低：41.8 µs	总参数最大；A-Net 需先做静态图优化才可能反超
参数 / 内存受限	共享主干 + 任务头	简单任务 4,742 参数，仅为独立基线 35.3%	复杂任务准确率降至 94.49% 且跨种子波动较大；主干参数仍全部参与前向

场景	首选方案	关键依据	注意事项
批量多任务服务	A-Net 硬路由	CPU batch=256：74.2 µs，3.39 M samples/s	小 batch 路由开销明显；GPU 小模型受 launch 限制，见第 5 节
任务复杂、差异大	A-Net	复杂任务 95.84%，独立基线 96.23%；A-Net 总参数仅 49.6%、激活参数 17.0%	总参数随任务数线性增长，需评估内存上限

4.3 局限性与后续工作

- **任务仍然是合成数据。**简单任务全部被拟合到 99% 以上；新增复杂任务将平均准确率拉低到 94.5–96.2%，但仍是公式生成、2% 标签噪声的二维数据。后续应加入真实表格/图像任务、多分类、更强噪声和更复杂的任务关系。
- **随机种子数量仍然有限。**简单任务只使用 seed=42，复杂任务使用了 3 个种子。更严谨的结论应统一使用至少 5 个种子，报告均值、标准差并做配对显著性检验。
- **GPU 只验证了单卡、eager、受控 batch 场景。**已在 RTX 5060 Laptop 上验证 CUDA 兼容性，但尚未测试多卡、CUDA Graph、TensorRT、动态 batching 和并发服务；CPU 侧也只测试了 eager 单线程，静态图会改变 batch=1 的结果。
- **A-Net 使用已知 Task ID 路由，不是学习得到的门控。**这保证了“只计算一个专家”，但缺少负载均衡和未知任务处理机制；扩展时需考虑任务数增长、专家容量和冷启动。
- **延迟不含工程链路。**真实服务的网络、序列化、特征预处理、排队和监控开销通常远大于模型本身的 40–90 µs（CPU）或 70–90 µs（GPU），部署前应做端到端压测。

5 CUDA 兼容性补充验证（RTX 5060 Laptop）

为确认实现不只适用于 CPU，本实验在本机 NVIDIA GeForce RTX 5060 Laptop GPU 上进行了实际 CUDA 验证。测试使用临时安装的 PyTorch 2.14.1+cu130（CUDA 13.0 轮子），驱动为 616.92（CUDA UMD 13.4）；GPU 显存 8 GB，桌面应用已占用约 2.3 GB。

5.1 环境与兼容性

表 6 CUDA 测试环境

项目	实际值
GPU	NVIDIA GeForce RTX 5060 Laptop GPU（7.93 GiB，26 SM）
Compute Capability	12.0（sm_120）
NVIDIA 驱动	616.92 / CUDA UMD 13.4

项目	实际值
PyTorch / CUDA	2.14.1+cu130 / 编译 CUDA 13.0
Python	3.14.6
cuDNN	9.24.0
PyTorch 编译架构	sm_75, sm_80, sm_86, sm_90, sm_100, sm_120
BF16 / TF32	BF16 支持；TF32 默认关闭

PyTorch 编译架构中包含 sm_120，与 RTX 5060 的 Compute Capability 12.0 完全匹配，不需要依赖 PTX JIT 回退。所有测试均正常通过，未出现 CUDA DLL 缺失、驱动不匹配或 OOM。

表 7 CUDA 兼容性测试结果

检查项	覆盖内容	结果
基础运行时	显存分配、CPU↔GPU 数据搬运、同步	PASS
数值精度	FP32 512×512 矩阵乘法，相对误差 2.27×10^{-7}	PASS
FP16 / BF16	半精度矩阵乘法，有限值与 dtype 检查	PASS
cuDNN	Conv2d forward + backward	PASS
三种模型	forward / backward / Adam step	PASS
A-Net 路由	仅 expert0 收到梯度，其他专家梯度为 None	PASS
混合精度	FP16 autocast forward + loss	PASS
端到端	5 epoch 快速验证 + 50 epoch 完整训练	PASS

显存峰值为 0.06 GiB allocated / 0.07 GiB reserved，8 GB 显存完全没有压力。A-Net 的硬路由在 GPU 上也严格保持“只执行一个专家”的语义：反向传播后只有被选中的 expert0 拥有梯度。

5.2 GPU 正确性与训练

完整 50 epoch 主实验可以在 GPU 上无修改跑通，且测试集准确率与 CPU 完全一致：独立基线 99.50%、共享主干 99.42%、A-Net 99.37%。说明当前实现没有因为设备迁移而改变模型行为。

但训练速度没有优势：GPU 上独立基线约 19.8 s、共享主干约 3.5 s、A-Net 约 3.5 s；同机 CPU 单线程分别约 2.9 s、1.6 s、1.1 s。独立基线作为第一个模型还包含 CUDA 上下文和算子首次加载的冷启动成本。对于每步只有 128×2 输入的微型网络，数据搬运、kernel launch 和 Python 调度远大于实际计算，GPU 不适合这个训练负载。

5.3 GPU 推理性能

表 8 GPU 推理延迟与吞吐

方案	batch=1 P50	batch=1 吞吐	batch=256 吞吐	batch=4096 吞吐
独立基线	71.3 µs	9.6k/s	2.04M/s	40.2M/s
共享主干 +3 个线性头	73.2 µs	9.7k/s	2.03M/s	40.6M/s
A-Net 极简版（硬路由）	76.7 µs	9.0k/s	2.03M/s	37.4M/s

注：P50 为 CUDA Event 测量的单次 forward；吞吐按 wall time 计算；GPU 同时承载桌面应用，绝对延迟受系统负载影响。

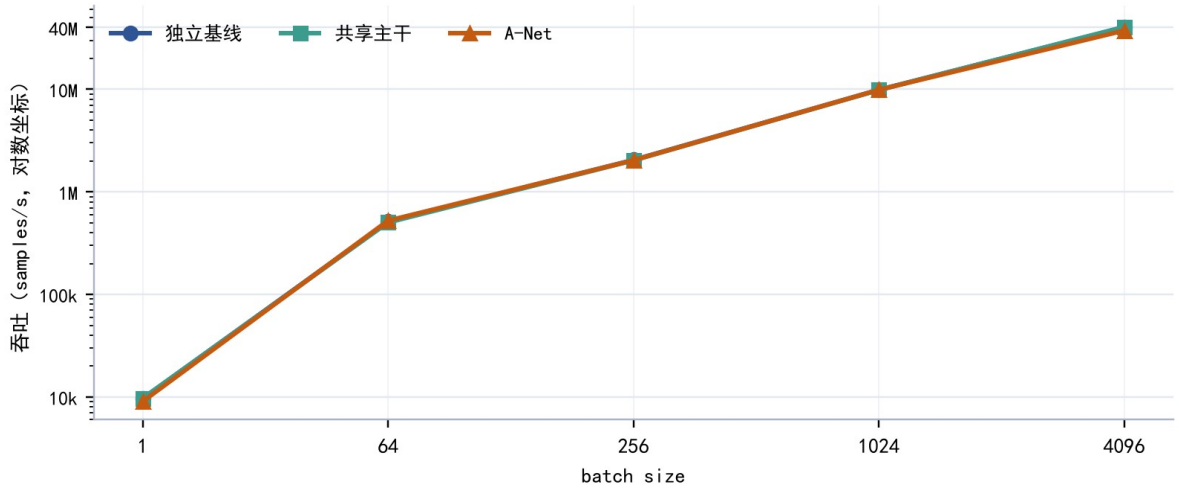


图 4 GPU 吞吐随 batch 的变化（对数纵轴）

batch=1：GPU 反而比 CPU 慢。RTX 5060 上单样本约 71–77 µs，而 CPU 单线程为 42–45 µs。模型太小，CUDA kernel launch 与 WDDM 调度占据了主要时间，真正的矩阵计算只有几微秒。

大 batch 才能体现 GPU 吞吐。batch 从 1 增大到 4096 时，三种模型的吞吐从约 9k samples/s 提升到 37M–41M samples/s；但单次 forward 的 P50 仍停留在约 67–77 µs，说明这个区间已经是 launch/调度瓶颈，而不是算力瓶颈。

A-Net 在 GPU 上没有额外优势。在 batch=4096 时，A-Net 吞吐约 37.4M samples/s，略低于独立基线的 40.2M 和共享主干的 40.6M。硬路由省下的专家矩阵计算，在这个极小模型上被固定的 GPU 调度成本淹没。

结论：代码已通过 RTX 5060 Laptop 的完整 CUDA 兼容性验证，可以正确训练和推理；但当前任务规模不足以发挥 GPU 优势。若要让 GPU 产生实际收益，应增大模型/隐藏维度、使用更大的 batch、采用 CUDA Graph 或 TensorRT 降低 launch 开销，并避免把单样本在线请求作为 GPU 的主要使用方式。

6 复杂任务泛化实验（CPU / GPU）

前面的三个任务都容易被拟合到 99% 以上，无法充分暴露架构差异。为此新增三个更复杂的二维二分类任务，保留 Task ID 多任务设定，同时加入 2% 标签噪声、更多样本和更宽的网络。所有任务使用 5,000 个训练样本、1,500 个测试样本和 3 个随机种子（2026/2027/2028），训练 80 个 epoch。

6.1 复杂任务设计

表 9 复杂任务定义

任务	标签规则	复杂化方式	难度来源
高频正弦	x2 > 0.55·sin(3x1) + 0.28·sin(7.5x1)	双频正弦叠加；2% 标签噪声；5,000 训练样本	边界高频振荡，局部样本稀疏
四同心环	4 个环带，标签按 0/1/0/1 交替；环带半径 [0.05,0.55]、[0.75,1.25]、[1.45,1.95]、[2.15,2.55]	每个环带等量采样；2% 标签噪声	非单调径向决策边界，需要多个决策面
旋转棋盘	25° 旋转后，按 floor((u+4)/1.1)+floor((v+4)/1.1) 的奇偶性分类	网格宽 1.1；2% 标签噪声	斜向多单元格边界，非轴对齐

复杂版同步扩大容量：独立基线为 3 ×（2-128-128-2），共 51,462 参数；共享主干为 2-128-128 主干加 3 个线性头，共 17,670 参数；A-Net 为共享 L1（2-128）+ 3 个专家（128-64-2），总参数 25,542，单样本激活参数 8,770。训练使用 Adam（lr=1e-3）、batch=256、80 epochs；CPU 侧为 8 线程，GPU 侧为 RTX 5060 Laptop。

6.2 CPU 与 GPU 结果

表 10 复杂任务测试准确率（CPU，3 seeds，mean ± std）

方案	任务 1 高频正弦	任务 2 四同心环	任务 3 旋转棋盘	平均
独立基线	96.31 ± 0.56	97.87 ± 0.20	94.51 ± 0.20	96.23 ± 0.19
共享主干	95.18 ± 0.37	95.16 ± 3.45	93.13 ± 0.87	94.49 ± 1.25
A-Net	95.58 ± 0.50	98.00 ± 0.13	93.93 ± 0.84	95.84 ± 0.14

表 11 复杂任务测试准确率（GPU，3 seeds，mean ± std）

方案	任务 1 高频正弦	任务 2 四同心环	任务 3 旋转棋盘	平均
独立基线	96.20 ± 0.61	97.87 ± 0.20	94.53 ± 0.18	96.20 ± 0.20
共享主干	95.24 ± 0.45	95.24 ± 3.62	93.11 ± 0.83	94.53 ± 1.36
A-Net	95.60 ± 0.55	97.98 ± 0.14	93.80 ± 0.85	95.79 ± 0.14

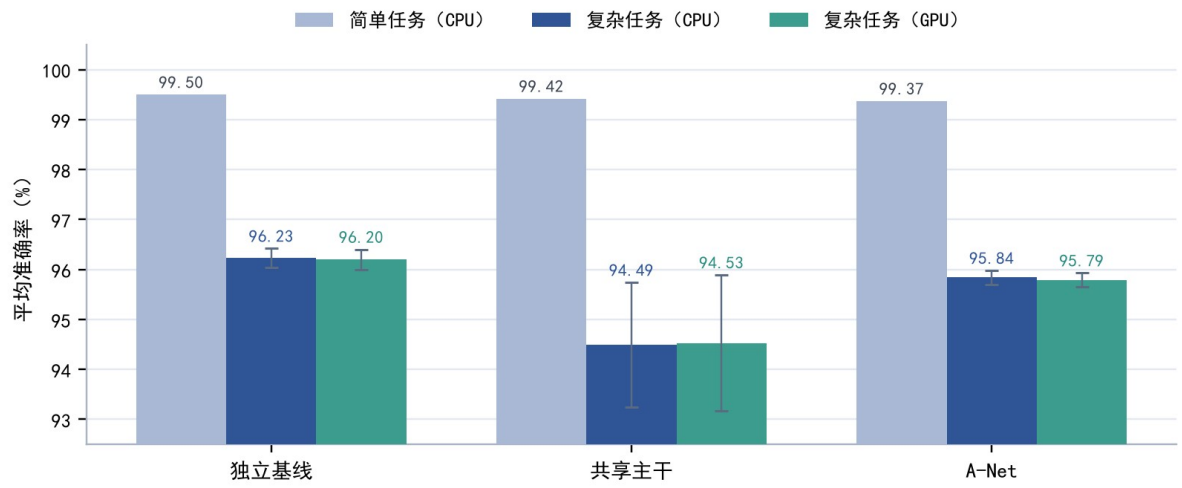


图 5 简单任务与复杂任务平均准确率对比（误差线为 3 seeds 标准差）

复杂任务明显拉开了差距。平均准确率从简单任务的 99.50/99.42/99.37% 降至 96.23/94.49/95.84%。独立基线仍然最高，但 A-Net 只低 0.39 个百分点；共享主干比独立基线低 1.74 个百分点，并且在四同心环任务上出现 91.2% 的低分种子，跨种子标准差达到 1.25。

A-Net 在复杂任务上的参数效率最好。A-Net 的总参数为独立基线的 49.6%，单样本激活参数只有独立基线的 17.0%，平均准确率却达到 95.84% 且标准差仅 0.14。这说明共享 L1 加上任务专属的非线性专家，在困难边界上比“共享主干 + 线性头”更能保留任务特化能力。

共享主干的瓶颈来自任务头容量。复杂任务需要多个非线性决策面，而共享主干的三个任务头只是线性分类器；所有任务被迫共用同一组 128 维特征，容易出现任务间折中。A-Net 的每个任务拥有完整的 128-64-2 专家，因此更稳定。

CPU 与 GPU 的准确率基本一致。三种模型在 CPU/GPU 上的平均准确率差异均不超过 0.05 个百分点（独立 -0.03、共享 +0.04、A-Net -0.05），说明 CUDA 路径没有引入精度问题。GPU 训练反而更慢：独立基线首次 24.8 s（冷启动）、共享 5.4 s、A-Net 5.6 s，CPU 8 线程分别为 3.7/3.1/2.9 s；仍是极小模型被 launch 开销支配的典型表现。

6.3 复杂任务结论

- **任务复杂度提升后，架构选择发生变化。**简单任务上三种结构都接近饱和；复杂任务上共享主干明显落后，而 A-Net 几乎追平独立基线，成为准确率/参数量折中的更优选择。
- **A-Net 的优势来自任务专属非线性专家，而不是总参数更少。**它的总参数仍高于共享主干，但单样本只激活一个专家；在复杂任务上，这部分额外存储换来了约 1.35 个百分点的平均准确率提升和更低的种子方差。
- **复杂任务仍未脱离合成数据范畴。**虽然比原任务更难，但仍由已知公式生成、只有 2% 标签噪声；后续应加入真实表格/图像任务、更强噪声和更多类别，验证结论的外部有效性。

附录 复现方式与指标口径

主实验脚本：multitask_anet_demo.py；复杂任务实验：complex_tasks_test.py；CPU 延迟基准：benchmark_latency.py；CUDA 兼容性/GPU 基准：cuda_compat_test.py。主实验与复杂任务实验均支持 --device auto|cpu|cuda，默认 auto；要复现文档中的 CPU 数据请显式使用 --device cpu。

```
pip install torch numpy
python multitask_anet_demo.py --device cpu
python benchmark_latency.py
python cuda_compat_test.py
python complex_tasks_test.py --device cpu
python complex_tasks_test.py --device cuda
python multitask_anet_demo.py --device cuda
```

表 12 指标口径

指标	定义
总参数量	模型所有 requires_grad=True 的参数元素总数，包含当前样本不会执行的专家。
单样本激活参数	一次 forward 中实际参与计算的参数元素数；A-Net = L1 + 1 个专家。
Top-1 准确率	测试集预测类别与真实标签一致的比例。
P50 / P95 延迟	纯模型 forward 耗时的第 50 / 95 百分位；batch 表值为整个 batch 的耗时。
GPU 延迟 / 吞吐	CUDA Event 的 forward P50；吞吐按 wall time 计算，包含 kernel launch 与 Python 调度。
吞吐	单位时间内处理的样本数，按样本数 / 总耗时计算。

说明：延迟测试使用随机初始化模型，因为没有加载训练权重不会改变稠密矩阵乘法的计算量和算子形状；文档中的准确率、参数量和延迟均在当前机器上实际运行得到。CPU 主实验使用 PyTorch 2.15.0+cpu，CUDA 验证使用 2.14.1+cu130；复杂任务 CPU 结果使用 8 线程，GPU 结果使用同一张 RTX 5060 Laptop。GPU 测试期间桌面应用同时占用显存和部分算力。换驱动、线程数、GPU 负载或运行时后，绝对值和排名都可能变化。

— 报告完 —